

DML/DDL, Cascade, Bits Conversion, Query Performance

Instructor: Aditya Parameswaran

Covers Lecture(s): 05–06

I DML/DDL

Aditya is trying to order boba for everyone! However, he needs some help with keeping track of boba orders. The student records for enrolled Data 101 students are stored in this table:

```
students (student_id, name, age, major)
```

1. Aditya wants to create an `orders` table that includes `student_id`, `student_name`, `order_id`, `order_info`, and `price`. **All orders should be from enrolled students in `students`.**

Write a SQL command to create this table, where `student_id`, `order_id`, and `price` are integers, and `student_name` and `order_info` are variable-length strings (30 and 50 characters max, respectively). Every order must belong to a student who exists in `students`; enforce this with a constraint.

```
CREATE TABLE orders(  
  student_id INTEGER, student_name VARCHAR(30), order_id INTEGER,  
  order_info VARCHAR(50), price INTEGER,  
  FOREIGN KEY (student_id) REFERENCES students (student_id)  
);
```

Explanation: We need to reference `student_id` as foreign key so if a student's `student_id` doesn't exist in `students` table anymore, a row using that `student_id` also won't be created since it's referenced as a foreign key

2. A particular student, Abra Cadabra, contacts course staff to change their boba order. Write a SQL query to update this student's order to 'Passion Fruit Green Tea, light ice, and large'.

```
UPDATE orders  
SET order_info = 'Passion Fruit Green Tea, light ice, and large'  
WHERE student_name = 'Abra Cadabra';
```

Explanation: We update the order for a specific student using the `WHERE` clause.

3. **[Optional]:** Suppose that, instead of the schema in Q1, `price` is stored as a `VARCHAR`. The `order_info` column contains boba flavor, ice level, and size. For example, the row (000, 'Aditya Parameswaran', 101, 'Brown sugar boba, light ice, large', '\$6') has boba flavor *brown sugar boba*. We want to group the order by boba flavors so it's easier to place the order. In addition, we want to calculate the total price for each flavor.

Write a SQL query that finds the total price and number of orders for each boba flavor.

```
SELECT flavor, COUNT(*), SUM(price)  
FROM  
(SELECT SUBSTRING(order_info, '^.*?'),) AS flavor, CAST(SUBSTRING(price, 2) AS  
  INTEGER) AS price
```

```
FROM orders)
GROUP BY flavor;
```

Explanation: We need to first cast the price to an integer and also extract the orders before counting them and summing their price.

II Cascade

4. In order to avoid over-ordering bubble tea, Aditya wants to make sure that if a student drop the class, their orders should automatically be dropped from the **orders** table. Alter the foreign key constraint in the **orders** table to implement this behavior.

```
ALTER TABLE orders
DROP FOREIGN KEY (student_id);
ALTER TABLE orders
ADD FOREIGN KEY (student_id) REFERENCES students(student_id)
ON DELETE CASCADE;
```

Explanation: By using the **ON DELETE CASCADE** property with the foreign key, any deletion in the **students** table that concerns a specific **student_id** will automatically result in the deletion of matching rows in the **orders** table.

Note: Any attempt to **UPDATE** the **student_id** in the **students** table should ideally throw an error as it's conceptually incorrect to change a student's ID. Such an update could have cascading impacts and unintended consequences.

5. A lucky student, Alex, decides to order themselves not one, but two bobas. The SQL commands are executed like so:

```
INSERT INTO orders (student_id, student_name, order_id, order_info, price)
VALUES (123, 'Alex', 101, 'Mango Boba, regular ice, medium', 5);
```

```
INSERT INTO orders (student_id, student_name, order_id, order_info, price)
VALUES (123, 'Alex', 102, 'Matcha Boba, no ice, large', 6);
```

Does the insertion of the second boba order for Alex violate the foreign key constraint of the **orders** table? Why or why not?

No, the insertion of the second order does not violate the foreign key constraint. The foreign key constraint on **student_id** in the **orders** table only requires that every **student_id** in **orders** exists in **students**. It does not ensure uniqueness in the **orders** table; hence, a student can have multiple rows (orders) in the **orders** table. If we wanted at most one order per student, we could add **UNIQUE (student_id)**.

Comment: Foreign keys are used to ensure referential integrity between tables in a relational database. They establish a link between the data in two tables. The main characteristic of a table with a foreign key is that it references the primary key of another table.

III Memory/Bits Conversion

Suppose each record in your database is 128 bytes, and you have 10 million records. Assume each 8kB page with records has 256 bytes of metadata.

6. Without considering metadata, how large is your database, in SI units?

Calculate the total data for records:

$$\text{Total data for records} = 128 \text{ bytes} \times 10,000,000 = 1,280,000,000 \text{ bytes}$$

Convert this to gigabytes in SI units:

$$\text{Since } 1 \text{ GB} = 10^9 \text{ bytes, converting bytes to GB: } \frac{1,280,000,000}{10^9} = 1.28 \text{ GB}$$

7. Considering metadata, how many pages do you need to store your database?

First, calculate the usable space per page by subtracting the metadata size from the total page size:

$$8kB = 8,000 \text{ bytes}$$

$$\text{Usable space per page} = 8,000 \text{ bytes} - 256 \text{ bytes} = 7,744 \text{ bytes}$$

Next, determine the number of records per page:

$$\text{Records per page} = \frac{7,744 \text{ bytes}}{128 \text{ bytes/record}} \approx 60 \text{ records per page}$$

Finally, calculate the total number of pages needed to store all records:

$$\text{Total number of pages} = \frac{10,000,000 \text{ records}}{60 \text{ records/page}} \approx 166,667 \text{ pages}$$

IV Query Performance

For this section, we recommend you open DataHub so you can try running these queries yourself in PostgreSQL. For your convenience, we also list the questions here.

This question looks at the impacts of aggregation and sorting on query performance.

8. Write a query that returns the actor names and the number of times the corresponding name appears in the `actors` relation.

```
SELECT COUNT(*), name FROM actors GROUP BY name;
```

9. Write a query that returns the actor IDs and the number of times the corresponding ID appears in the `actors` relation.

```
SELECT COUNT(*), id FROM actors GROUP BY id;
```

10. Run `EXPLAIN ANALYZE` on your two queries above. Why do you think the `name` query use a Sequential Scan, whereas the `id` query use an Index Only scan?

Because only the `id` data exists in the index, and Index Scan is less I/O-intensive than Sequential Scan when available. For the `name` column, Sequential Scan is the only possible way to retrieve its data.

11. Write a command that creates an index `name_actor_index` on the `name` attribute of the `actors` table. Rerun your `EXPLAIN ANALYZE` of your Question 8 query on `name`. Why does the `name` query now use an Index Only Scan? What index is it using?

```
CREATE INDEX name_actor_index on actors (name);
```

The query can now make use of the newly created `name_actor_index` index, since an Index Scan incurs less I/O cost than a full Sequential Scan. Notice that this is only possible because we already paid the I/O cost upfront when creating this index.

12. Now consider sorting. Modify your Question 9 query so the results are ordered by ID (sorted by lowest ID first), and run `EXPLAIN ANALYZE` on it. Do you expect this to take longer than the unsorted version? Why or why not?

```
EXPLAIN ANALYZE SELECT COUNT(*), id FROM actors GROUP BY id ORDER BY id;
```

You might be surprised to find out that this query did not take any longer than the previous query. In fact, the query plan did not even include an additional sorting step. Why? Because the B+ tree index already provides ordering “for free”. When you traverse a B+ tree index to retrieve all the ID values, you already get them back in a sorted order.